

Plan → mission → report

Hardware requirements, input, user interactions and output for connecting your drone and ground station to AutoAssess.

YOU LEAVE WITH

What your side must provide, what you receive, and a working Python loop in the sandbox

BRING

A laptop with Chrome and your Fusion login



The hour

09:15 5' Why we're here, and the one rule: **CDF is the only integration point**

09:20 10' System and mission lifecycle

09:30 10' Hardware and environment requirements

09:40 8' Input: the plan and the reference map

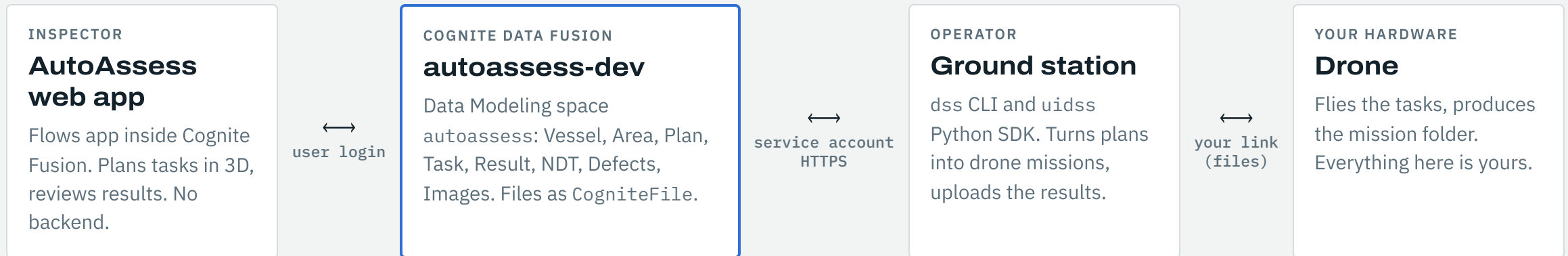
09:48 5' User interactions: planner, operator, reviewer

09:53 7' Output: what the mission must upload

10:00 12' Sandbox: run the flow in your browser with `sim_drone`

10:12 3' Open questions we need your answer on

Four parts, one integration point



The web app and the ground station never talk to each other. Everything goes through CDF. Your integration surface is **one JSON file in** and **one mission folder out**.

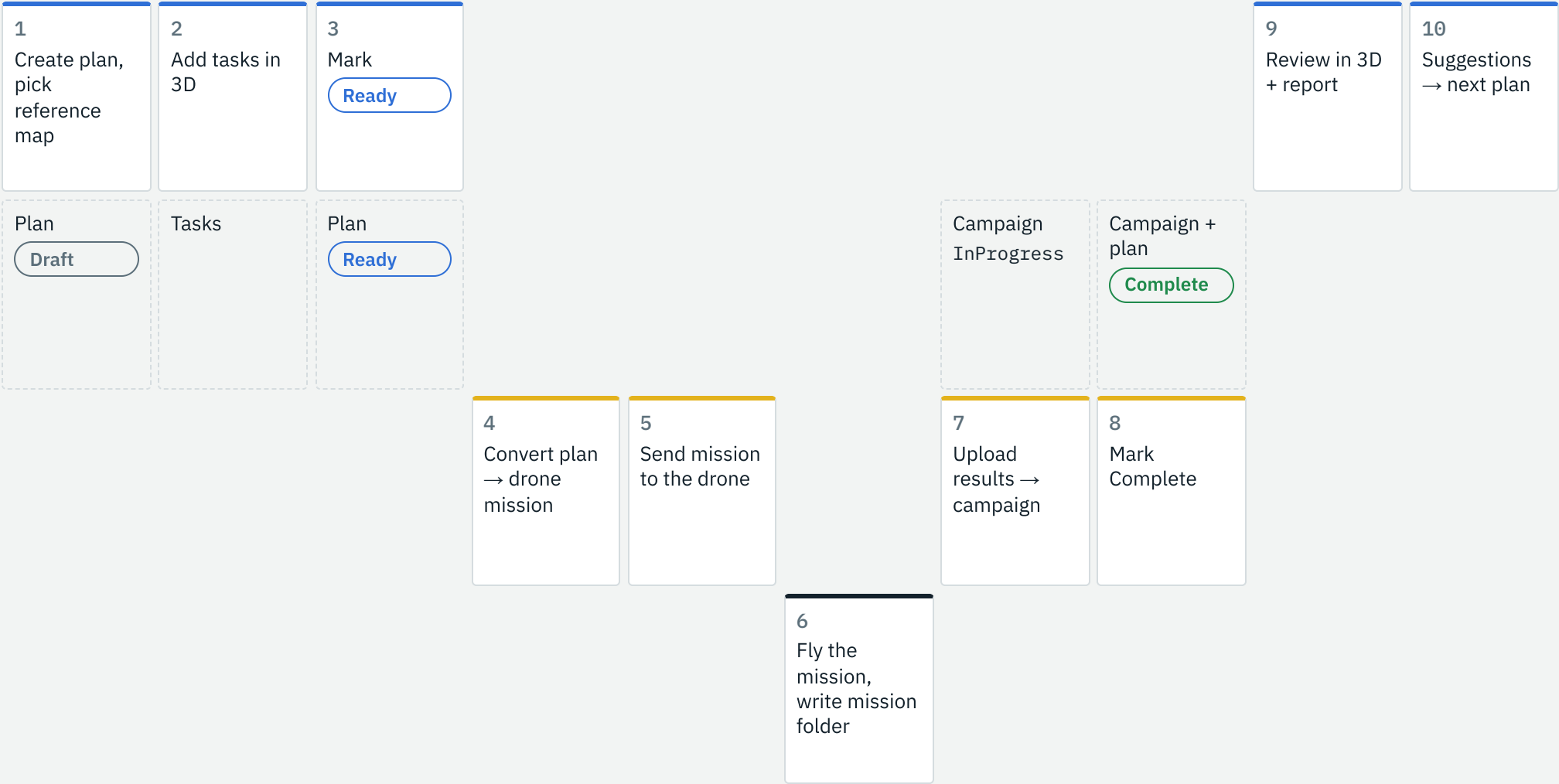
Mission lifecycle

Inspector
web app

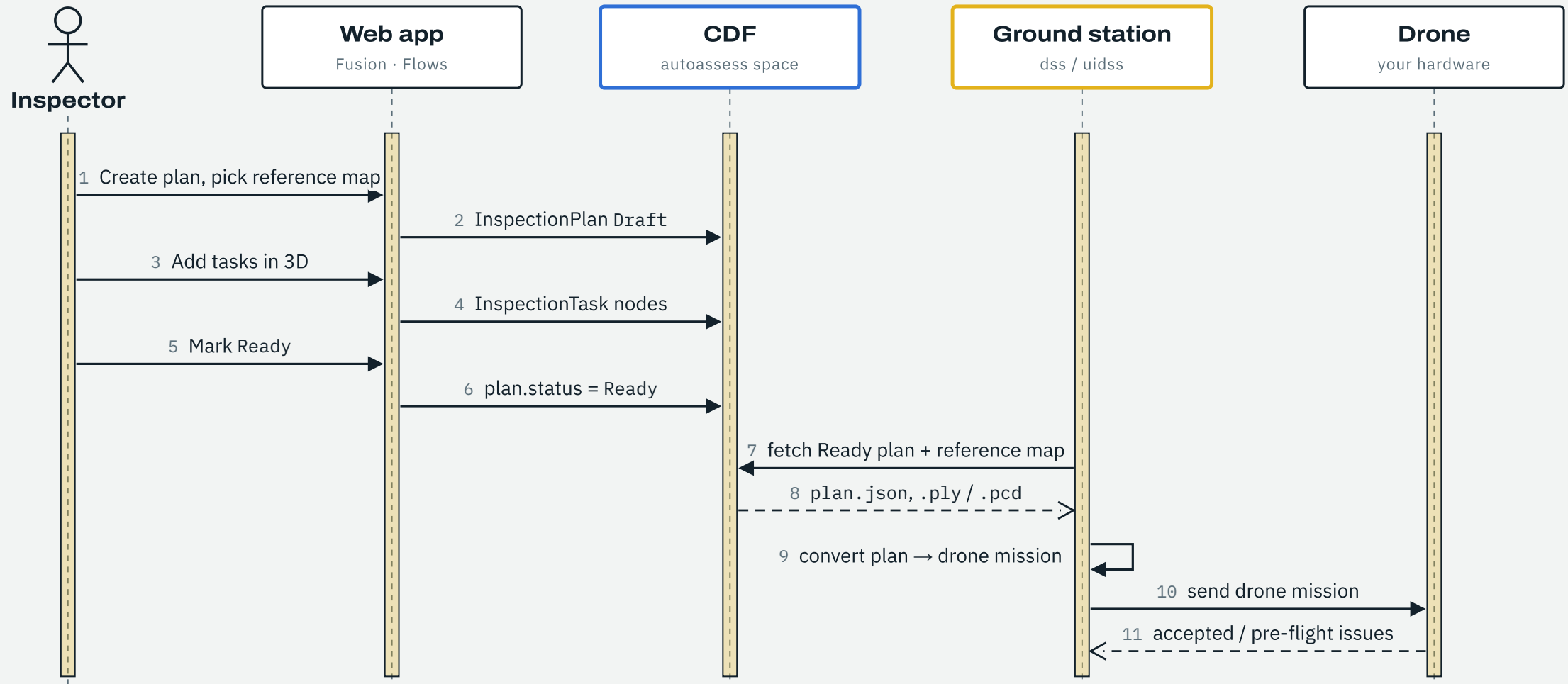
CDF
autoassess space

Operator
ground station

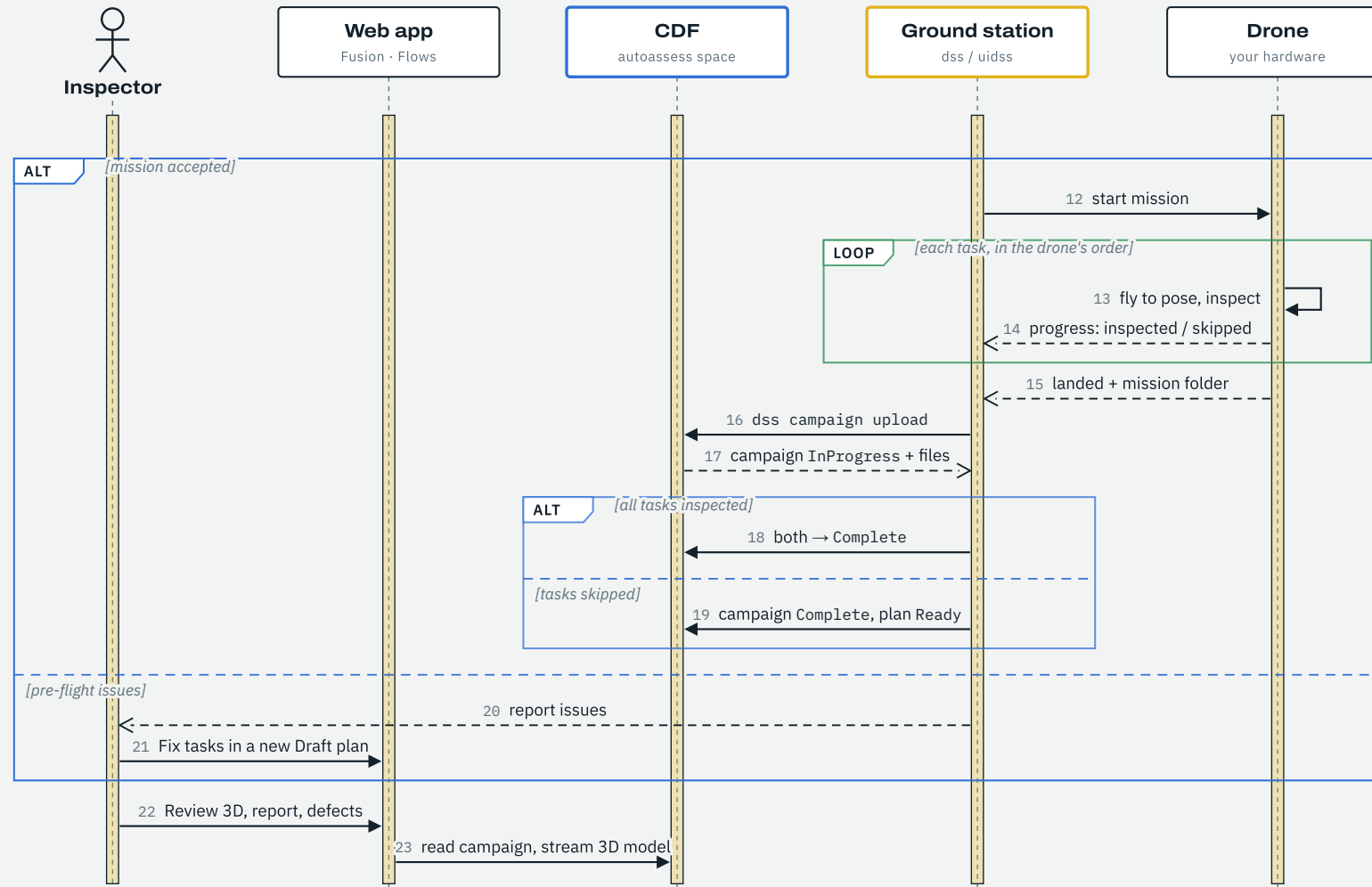
Drone
your hardware



Sequence 1/2: plan and hand-over



Sequence 2/2: fly, upload, review



Hardware and environment

Ground station CONFIRMED

- Python ≥ 3.12 with uv; macOS or Linux
- Outbound HTTPS (443) to westeurope-1.cognitedata.com and the login provider
- CDF service-account credentials in sdk/.env, never on the drone
- Disk for maps and meshes: a single mesh can be 100–270 MB
- CPU for the 3D model build during upload (a few minutes locally)

Drone and sensors DERIVED

- Localise in the **reference map frame** of the plan's mapExternalId
- 3D capture: mesh (PLY, per-vertex RGB) and/or point cloud (PCD)
- RGB camera with known intrinsics and body→camera extrinsic
- Pose log at image timestamps (TUM format)
- UT thickness gauge for ndt_thickness tasks

Accuracy and reviewers

FROM
THE PRD

- **KO-5:** < 1 cm relative accuracy of defect/NDT registration
- **NFR-04:** always cloud-connected at upload time, no offline mode
- **NFR-05:** reviewers need Chrome/Edge/Firefox with WebGL 2
- **KO-7:** demonstrated on bulk carrier, container ship and tanker

TBD PARTNER

Onboard compute · link between drone and ground station (tether, Wi-Fi, SD card) · positioning accuracy achieved in a tank · who converts sensor output to PLY/PCD/TUM

Input: plan.json

```
{
  "planExternalId": "plan-6c1e...",
  "name": "Tank 3 follow-up",
  "areaName": "BWT 3P",
  "mapExternalId": "result-9ab2...", // frame of all coordinates
  "tasks": [
    { "id": "task-...", "kind": "element",
      "inspectionType": "visual",
      "targetElement": { "type": "longitudinal",
        "center": [4.17, 0.24, 0.84] } },
    { "id": "task-...", "kind": "region",
      "inspectionType": "ndt_thickness",
      "position3d": [2.20, 1.09, 0.92],
      "normalVector": [0.0, -1.0, 0.0],
      "radiusM": 0.3 }
  ]
}
```

Tasks are an unordered set

The drone decides the order. The sandbox uses nearest-neighbour as an example.

Two kinds of task

element: inspect a structural element at its centre point. **region**: a surface point with outward normal and radius.

Plus the reference map

dss plan download-map fetches the .ply/.pcd of the mapExternalId campaign to localise against.

User interactions

INSPECTOR · BEFORE

Plans the mission

Fusion → AutoAssess

1. Open vessel → area → 3D viewer
2. Create a plan, choose the reference map
3. Click elements or surfaces, add as tasks
4. Mark the plan **Ready**

OPERATOR · ON SITE

Runs the mission

ground station terminal

1. Convert the Ready plan into a drone mission (plan + reference map)
2. Send the mission to the drone and start it
3. Upload the mission folder: dss campaign upload
4. Mark campaign and plan **Complete**

REVIEWER · AFTER

Reviews the result

Fusion → AutoAssess

1. Toggle mesh, point clouds, NDT, images per campaign
2. Campaign report: stats, NDT table, images
3. Triage defects
4. Accept suggestions → next Draft plan

Output: the mission folder

FILE	BECOMES IN CDF	FORMAT NOTES
*.ply	Mesh on the campaign, then a streamed 3D model	Binary or ASCII; per-vertex RGB (camera colour), optional face colours (segments)
*.pcd	Point cloud on the campaign, with a label	PCL PCD; a label field colours points
*.csv	NdtMeasurement nodes	timestamp,thickness,x,y,z: timestamp in ns , thickness in metres
ssg.yaml	StructuralElement nodes on the area	Scene graph: manhole, longitudinal, wall, compartment
metrics.yaml	CampaignMetric nodes	Decimal or percentage values
tum/	DroneImage nodes + PNG files	rgb.txt, groundtruth.txt (tx ty tz qx qy qz qw), sensor.yaml

All coordinates in the reference map frame. Upload creates a campaign (InProgress), then you mark it and the plan

Complete

.

Reality check: how do tasks reach the drone?

Confined-space inspection drones such as Flyability Elios 3 and ScoutDI Scout 137 are flown by a pilot. We found no public API for uploading waypoints to either.

- Elios 3 data goes to Flyability Cloud, which offers REST APIs for the recorded data.
- Scout 137 tags data to its position on a 3D map and uploads to Scout Portal after flight.
- So `plan.json` may be a **pilot checklist** plus a post-flight import, not an autopilot program.

Which one is your drone?

A • Autonomous

Your software reads `plan.json` and calls something like `goto(pose) / inspect()`. The sandbox's `sim_drone` is shaped for this.

B • Pilot-flown

Show tasks to the pilot, then convert the vendor export (cloud API, SD card) into the mission folder before `dss campaign upload`.

PLAN

FLY

UPLOAD

REVIEW

10:00

The Drone Sandbox

Python runs in your browser (Pyodide). No install, no credentials.

- uidss: the **same API** as the real ground-station SDK, reading the real project, read-only
- `sim_drone = SimDrone()`: a simulated drone you drive with poses
- Task panel shows each task's state during the flight
- Marking the plan Complete is **simulated**. Nothing is written to CDF.

OPEN IT

<https://cog-autoassess.fusion.cognite.com/autoassess-dev/custom-apps/development/autoassess-drone-sandbox/3010?cluster=westeurope-1.cognitedata.com&workspace=flows>

AutoAssess Drone Sandbox
Python in your browser with the uidss SDK: pick up inspection plans, check them, send off a simulated drone.

CDF project autoassess-dev (read-only) 16 plans · 7 areas Reload Python

3. Fly the mission Run Stop Reset

```

1 """3. Fly the mission – pick up the newest Ready plan, fly it with the s
2 import json
3 from pathlib import Path
4
5 from dss_sandbox import SimDrone
6 from uidss import UidssClient
7
8 client = UidssClient.from_env()
9
10 # 1. Newest Ready plan that has tasks, across all vessels/areas.
11 candidates = [
12     (area, p)
13     for vessel in client.vessels.list()
14     for area in client.areas.list(vessel.space, vessel.external_id)
15     for p in client.plans.list(area.space, area.external_id)
16     if p.status == "Ready"
17 ]
18 counts = client.plans.count_tasks([p.external_id for _, p in candidates])
19 candidates = [(area, p) for area, p in candidates if counts[p.external_id]
20 ]
21 if not candidates:
22     raise SystemExit("No Ready plan with tasks found. Add tasks to a plan
23 area, plan_meta = max(candidates, key=lambda c: c[1].created_time)

```

Console Clear

```

t= 14.5s Arrived at (2.91, 0.29, 2.15), yaw -9°
t= 17.5s Inspected task-0853c1cd-457e-4835-b221-8f82393acb16 (visual, 3 s)
t= 17.5s Returning home
t= 28.9s Landed

Mission plan-8018715f-b235-411c-b850-a2185c1bf9e5: 2/2 tasks inspected, 11.5 m flown in 29 s (simulated)
simulated: plan plan-8018715f-b235-411c-b850-a2185c1bf9e5 → Complete (not written to CDF)

✓ Finished in 44 ms

```

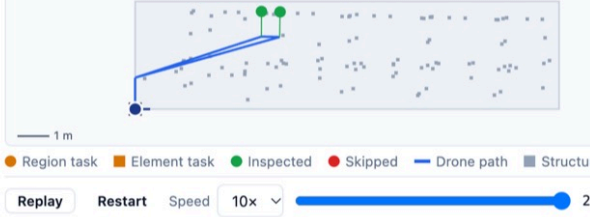
Drone simulator · plan-8018715f-b235-411c-b850-a2185c1bf9e5
Test Area · speed 0.5 m/s · battery 900 s

TOP VIEW (X-Y)



1 m

SIDE VIEW (X-Z)



1 m

● Region task ● Element task ● Inspected ● Skipped — Drone path ■ Structural

Replay Restart Speed 10x 28

Tasks · plan-8018715f-b235-411c-b850-a2185c1bf9e5 simulated

Task ID	Region	Visual	Status	Time
task-0853c1cd-457e-4835-b221-8f82393acb16	region	visual	inspected	at 17.5s
task-5542b775-25a1-4fdb-a197-78e614c7762a	region	visual	inspected	at 17.5s

Mission summary

Metric	Value
Tasks inspected	2 / 2
Distance	11.5 m
Simulated time	29 s
Skipped	0

Mission log

```

0.0s Take-off from (-1.63, 0.18, -0.14)
10.3s Arrived at (2.34, 0.25, 2.16), yaw -72°
13.3s Inspected task-5542b775-25a1-4fdb-a197-78e614c7762a (visual, 3 s)
14.5s Arrived at (2.91, 0.29, 2.15), yaw -9°
17.5s Inspected task-0853c1cd-457e-4835-b221-8f82393acb16 (visual, 3 s)
17.5s Returning home
28.9s Landed

```

Step 4 in the sandbox: fetch and check the plan

```
from uidss import UidssClient

client = UidssClient.from_env()
for vessel in client.vessels.list():
    for area in client.areas.list(vessel.space, vessel.external_id):
        for p in client.plans.list(area.space, area.external_id):
            print(p.status, p.name, p.external_id)

# exactly what 'dss plan download' writes
client.plans.download(p.space, p.external_id,
                      area.name, Path("plans/p.json"))

# your checks: pose present, inside the area,
# unit normal, reachable...
```

Console

[Clear](#)

```
[FAIL (3)] BWT 3P / Draft - needs review [Draft] {'element/visual': 2,
'region/visual': 1, 'region/ndt_thickness': 1}
- demo-plan-draft-task-1: no pose
- demo-plan-draft-task-2: [14.5, 0.2, 1.0] is outside the area bounds
- demo-plan-draft-task-3: normalVector missing or not unit length
[PASS] BWT 3P / Tank 3 follow-up [Ready] {'element/visual': 4,
'element/ndt_thickness': 1, 'region/ndt_thickness': 2, 'region/visual': 1}
[PASS] BWT 3P / NDT sweep - port side [Ready] {'region/ndt_thickness': 6}
[PASS] Cargo Hold 2 / Hold 2 visual [Complete] {'element/visual': 3}

✓ Finished in 22 ms
```

Example 2 on demo data: a broken Draft plan fails three checks. On the real project it flags a Draft task outside its area bounds.

Steps 5–6: the mission, flown with poses

```
from dss_sandbox import (SimDrone, pose_for_task,
                          BatteryLowError)

sim_drone = SimDrone(speed_mps=0.5, max_flight_time_s=900)
issues = sim_drone.load_plan(plan)  # pre-flight checks

sim_drone.takeoff(height_m=1.0)
for task in plan["tasks"]:          # you pick the order
    # Pose(x, y, z, roll, pitch, yaw), map frame
    pose = pose_for_task(task, standoff_m=0.8,
                          approach_from=sim_drone.pose)

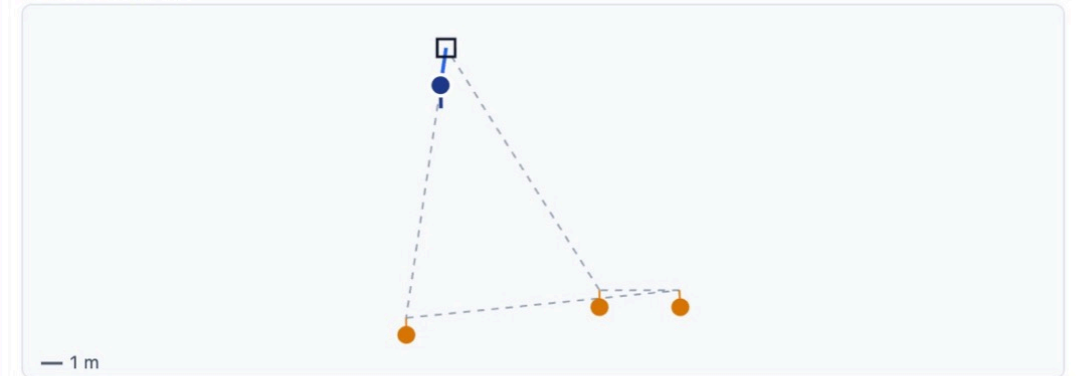
    try:
        sim_drone.goto(pose)
        sim_drone.inspect(task["id"])
    except BatteryLowError:
        break
sim_drone.return_home()
sim_drone.land()
print(sim_drone.report().summary())
```

Drone simulator · plan-9090c575-8ea3-4464-963b-727468be1cb8

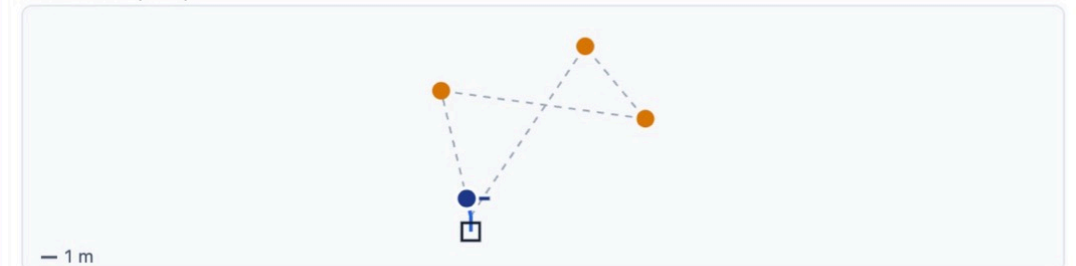
● Flying

Simulated Area · speed 0.5 m/s · battery 900 s

TOP VIEW (X-Y)



SIDE VIEW (X-Z)



● Region task ● Element task ● Inspected ● Skipped — Drone path ■ Structural element

Pause

Restart

Speed

10x

v

6.2 / 115 s

The task panel moves each task through pending → en route → inspecting → inspected (or skipped, with the reason).

Steps 7–8: upload and close the plan

Ready → mission → Complete

After landing, the example calls `client.plans.update_status(..., "Complete")`. In the sandbox this only changes the local copy, and the console says *not written to CDF*.

On the ground station, `dss campaign upload` runs first: the mission folder becomes a campaign, and then the campaign and the plan are marked Complete.

Tasks have no status in the data model. Per-task progress lives in the drone, and in the sandbox's task panel.



What you implement, what the sandbox fakes

CALL	ON YOUR DRONE	IN THE SANDBOX
<code>load_plan(plan)</code>	Parse <code>plan.json</code> , run your own pre-flight checks	Checks pose, bounds, normals; returns issues
<code>takeoff(height_m)</code>	Arm and climb	Climbs 1 m from the take-off point
<code>goto(Pose)</code>	Fly to x, y, z in the map frame, hold roll/pitch/yaw	Straight line at constant speed; battery check
<code>inspect(task_id)</code>	Capture images, take the UT reading, log the pose	Hover 3 s visual / 8 s NDT; task → inspected
<code>return_home()</code> , <code>land()</code>	Return and land; write the mission folder	Flies home, lands, builds the report
<code>on_event(cb)</code> , <code>report()</code>	Stream progress to the operator	Events, task panel, mission summary
<code>plans.update_status(...)</code>	Real <code>uidss</code> call after <code>dss campaign upload</code>	Simulated; nothing written to CDF

Hands-on (5 min): open example 4, change the order or the stand-off, and make the drone skip a task on purpose. Then write the same loop against your own drone's SDK.

We need your answers

- **OQ-5:** is the `plan.json` task shape what your drone can consume?
- **Plan status:** do you need an `InProgress` state on the plan, or is the campaign enough?
- **OQ-10:** is an element's centre point enough to plan a viewpoint, or do you need its extents?
- **Hardware:** onboard compute, drone ↔ ground-station link, positioning accuracy in a tank
- **Formats:** can you produce PLY/PCD/TUM directly, or who converts the vendor export?

TUTORIAL

`docs/tutorial/` in the repo

GROUND-STATION SDK

`sdk/` • `uv run dss --help`

SANDBOX

<https://cog-autoassess.fusion.cognite.com/autoassess-dev/custom-apps/development/autoassess-drone-sandbox/3010?cluster=westeurope-1.cognitedata.com&workspace=flows>